

Software Lifecycle Management



Fachbereich Elektrotechnik
und Technische Informatik

Dr. rer. nat. Nils Beckmann
nils.beckmann@th-owl.de

Vorgehensmodelle: Agil

Themen

- Agile Vorgehensmodelle
 - Beispiele
 - Evolutionäre Entwicklung
 - Explorative Entwicklung
vs. Wegwerf-Prototypen
 - Iterative Entwicklung
 - Inkrementelle Entwicklung
 - Extreme Programmierung (XP)
 - Scrum

Vorgehensmodelle: "Klassisch" oder "Agil"

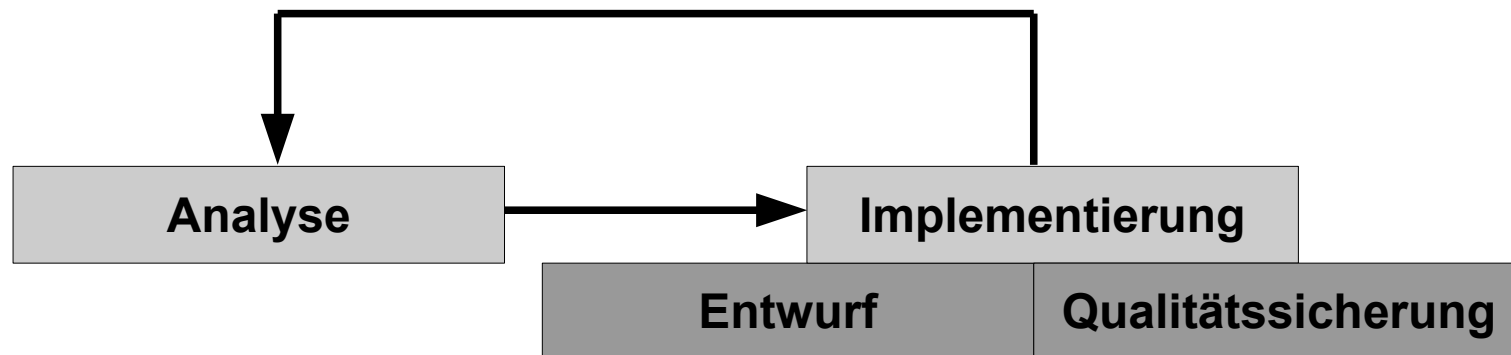
	Klassisch	Agil
Dauer von Iteration/Zyklus	Lang	Kurz
Planung	Umfangreich (mehr „Bürokratie“)	Eher kürzer
Historie	Älter (etwa ab 1950er)	Jünger (etwa ab 1990er)
Umfeld, Markt, Anforderungen	Stabil	Wandelbar
Ziel, Vorteile	Durch gute Struktur ein fest vereinbartes Ergebnis sicher erreichen	Frühzeitige Darstellung und Erhalt von Ergebnissen und Prototypen

Beispiele für Agile Vorgehensmodelle

- Evolutionäre Entwicklung
(ist *agil* und *generisch*)
 - Iterative Entwicklung
 - Inkrementelle Entwicklung
- Extreme Programmierung (XP)
- Scrum
- Kanban
(ist *agil* und *generisch*)
 - Scrumban
- Feature Driven Development (FDD)
- usw.

Evolutionäre Entwicklung

- Vorgehen:
 1. Erstelle Anfangsimplementierung der Software
 2. Hole Rückmeldung vom Benutzer ein
 3. Verfeinere die Software **Iterativ** über viele Versionen hinweg (auch über Punkt 2.) und teile sie in **Inkremente** (Bausteine) auf, bis ein angemessenes System entstanden ist



Evolutionäres Vorgehen

A) "Explorative Entwicklung"

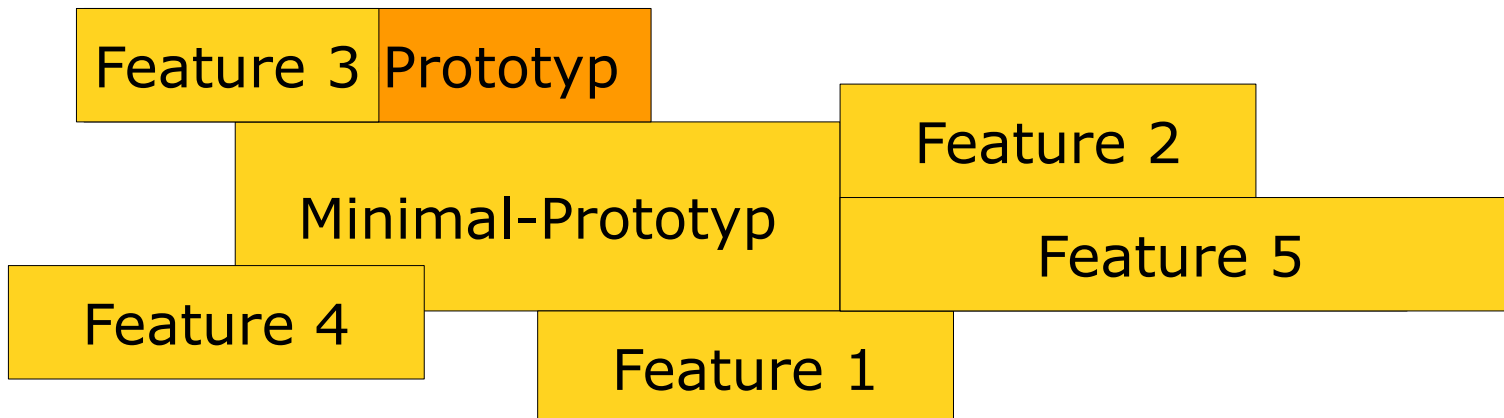
- Weitere Funktionalitäten werden in sukzessiver Abstimmung mit dem Kunden entwickelt und eingebaut

B) "Wegwerf-Prototypen":

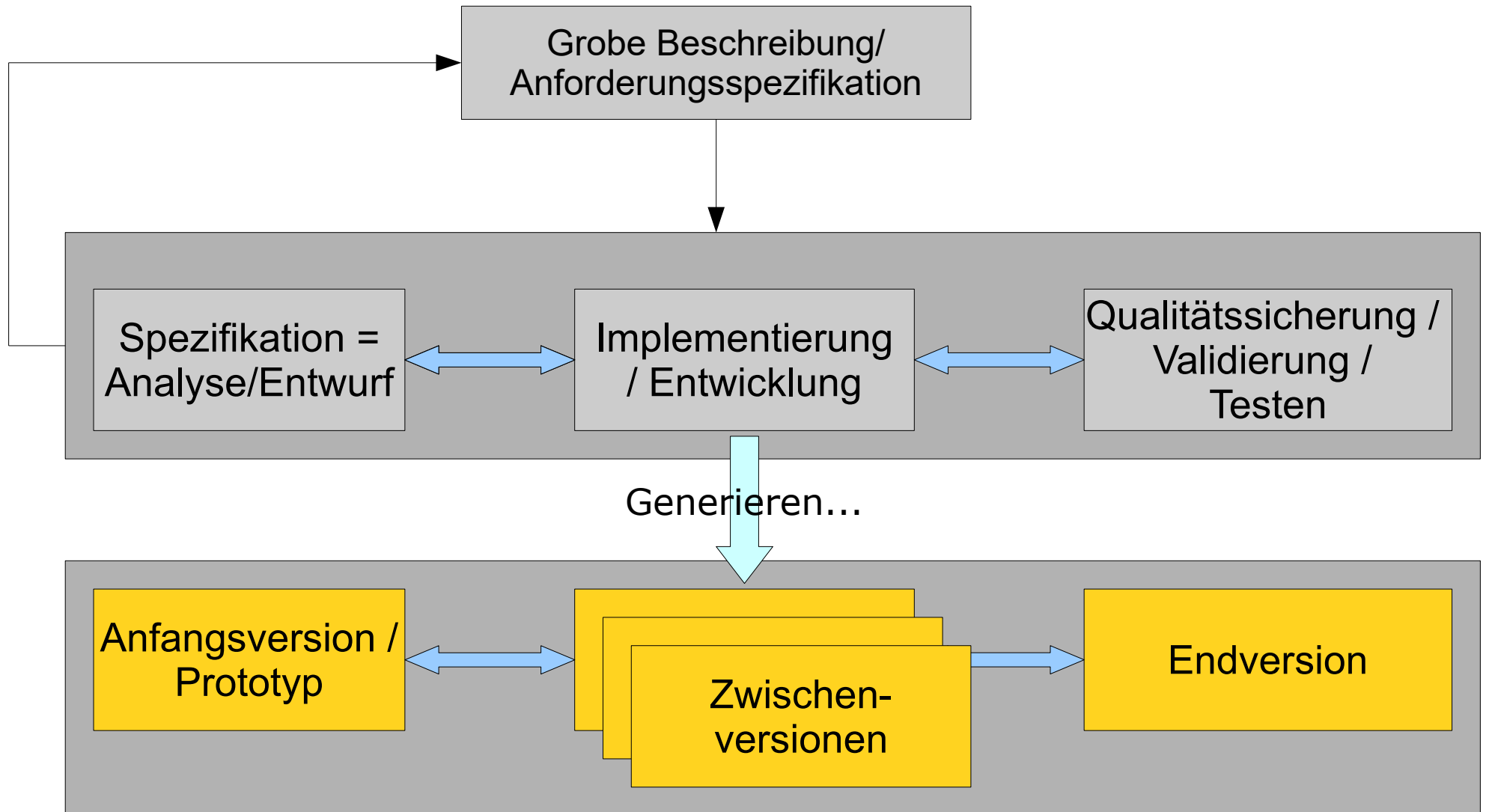
- Entwickle kleine Prototypen für (unklare) Kundenwünsche, die zur Demonstration genutzt werden und danach ggf. nicht weiter verwendet werden

Evolutionäre Ergebnisse

- Software-Produkt entsteht sukzessive
 - Weitere Funktionalitäten werden laufend hinzugefügt während des Prozesses
- Begonnen wird mit einem "Minimal-Prototyp"
 - = Eine lauffähige Software mit den minimal möglichen Bausteinen um grundlegend Funktionsweise und Design zu demonstrieren



Evolutionäre Entwicklung



Unterformen der Evolutionären Entwicklung

- Evolutionäre Entwicklung

- A) Fokus: **Iteration**

- "Iterative Entwicklung"

- Mehr Zeit wird auf die Rücksprache mit dem Kunden und engeren Austausch verwendet

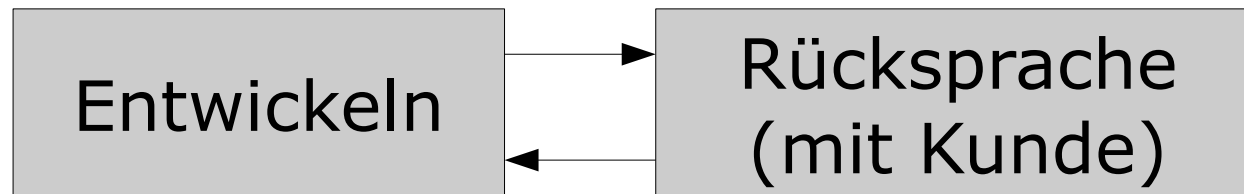
- B) Fokus: **Inkrement**

- "Inkrementelle Entwicklung"

- Mehr Zeit wird auf die Einigung auf bestimmte Produkt-Inkrement mit dem Kunden verwendet

Iterative Entwicklung

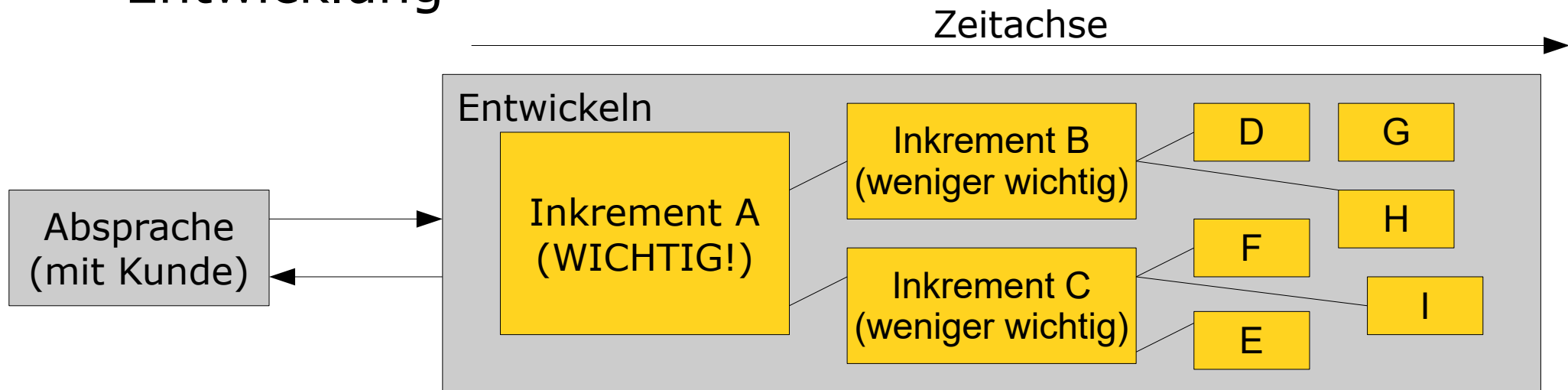
- Die "Iterative Entwicklung" ist eine Ausprägung/Unterform der Evolutionären Entwicklung



- Die Zyklen können kürzer (kleines Inkrement) oder länger (großes Inkrement oder Produkt) sein:
 - Unabhängig davon besteht dauerhafter Austausch mit dem Kunden über einzelne Entwicklungsfortschritte

Inkrementelle Entwicklung

- Die "Inkrementelle Entwicklung" ist eine Ausprägung/Unterform der Evolutionären Entwicklung



- Es erfolgt inkrementelle Absprache auf bestimmte Produkt-Inkremente mit dem Kunden
 - Ermöglicht Höhere Kohäsion, Losere Kopplung

Evolutionäre Entwicklung: Vorteile

- Unmittelbarere Erfüllung von Kundenwünschen
- Schnell erreichbar: Abliefern von Ergebnissen, Nutzen von Software, Vorzeigen von Prototypen
- Vollständige Spezifikation muss nicht fertig sein vor Implementierung $\Rightarrow$ Schneller Start mit Entwicklung möglich
- Neue Wünsche können angeregt und unmittelbar eingepflegt werden
- Dynamisches Anpassen an wandelnde Anforderungen ist über das Vorgehen machbar

Evolutionäre Entwicklung: Nachteile

- Prozess ist nicht sichtbar
 - Nachverfolgbarkeit ist schwierig
 - Fortschrittsmessung und Zeitplanung ist schwierig
- Systeme sind oft schlecht strukturiert
 - Architektur ist fehleranfällig
 - Kriterien für einen guten Entwurf sind schwieriger zu gewährleisten
 - Projekte ab einigen 100k Zeilen können auf unlösbare Probleme stoßen hinsichtlich Fehlerkorrektur oder Weiterentwicklung
- Integration der Ergebnisse mehrerer Teams schwierig

Extreme Programmierung (XP): Konzept

- Begriff 1999 von Kent Beck geprägt
- Steigerung der **Iterativen Entwicklung**: Noch stärkere Einbeziehung des Kunden in den Prozess
- Vorgehen:
 - Alle Anforderungen werden...
 - ...ausschließlich in **User Stories** ausgedrückt
 - ...und direkt implementiert
 - Ein Bevollmächtigter des Kunden ist *permanent* beim Entwicklungsprozess dabei und formuliert Abnahme- und Akzeptanzkriterien

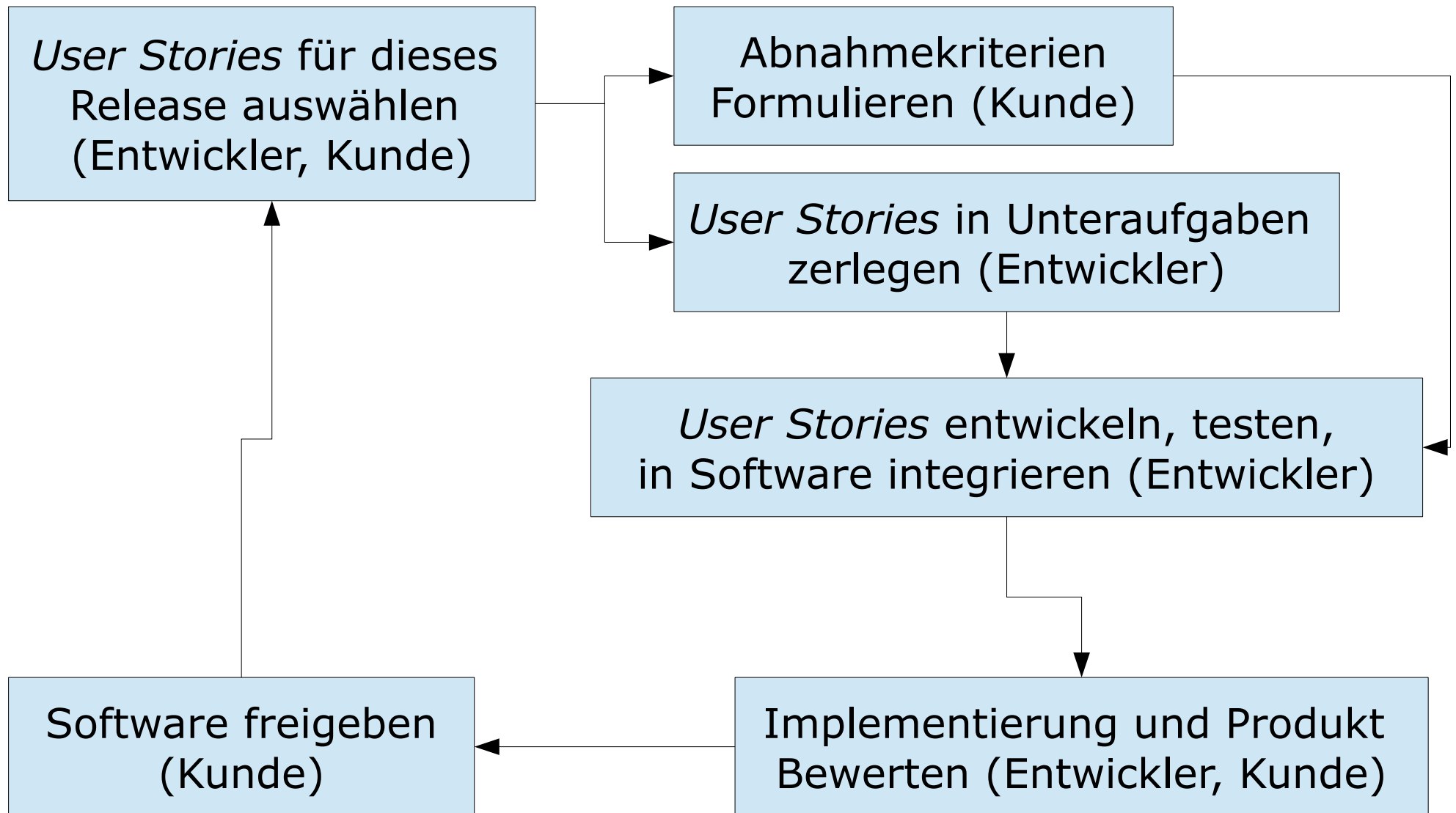
Extreme Programmierung (XP): Vorgehen

- Jede *User Story* in "Paarprogrammierung" bearbeiten
 - = Zwei Personen arbeiten gleichzeitig zusammen, eine programmiert, die andere schaut auf den Bildschirm und gibt Hinweise – nach einiger Zeit wird gewechselt; Implementierung besprechen
- Reihenfolge der Bearbeitung:
 - 1) Zerlegung der Aufgabe in Teilaufgaben
 - 2) Formulieren von Tests für alle Teilaufgaben
 - 3) Implementierung und Durchführen der Tests
 - 4) Einbindung des getesteten Codes in die Software

Extreme Programmierung (XP): Ideen

- Die Menschen sollen in den Vordergrund gerückt werden:
 - Indem paarweise programmiert wird
 - Zeiten der Arbeitsphasen kleinteiliger werden
 - Eigentum am Systemcode kollektiviert wird
- Erhoffte Wirkungen:
 - In kurzen Zeitabständen entstehen neue, kleine, häufige Releases
 - Der Kunde kann sich das neue Produkt sofort ansehen und Rückmeldungen geben (Prototypen und Inkremente)

Extreme Programmierung (XP): Ablaufdiagramm



Extreme Programmierung (XP): Vorteile und Nachteile

- **Vorteile:**

- Schnelle Erstellung von lauffähiger Software und Prototypen
- Unmittelbare Kundenrückmeldung bezüglich Akzeptanz und Änderungswünschen
- Unterstützt permanentes Refactoring (Code-Umstrukturierung)

- **Nachteile:**

- Nachverfolgbarkeit ("Traceability") kann schwierig sein
- Bei schneller Entwicklung kann es nicht kosteneffektiv umgesetzt werden, umfangreich zu dokumentieren
- Integration Ergebnisse verschiedener Teams kann schwierig sein
- Systeme können schlecht strukturiert sein
 - Bei größeren Projekten (ab ca. 100k Zeilen) wird man auf unabänderliche Probleme stoßen, die eine Weiterentwicklung schwierig bis unmöglich machen

Scrum

- Entwickelt in den 1990er Jahren
 - Scrum = "Gedränge"
- Grundgedanke ist, dass viele Projekte zu komplex seien um anfangs einen vollumfänglichen Plan zu erstellen
- Besser sei es, man geht wie folgt vor:
 - **Inkrementell:** Das Wichtigste zuerst
 - **Empirisch:** Auf das Entwickelte zurückblicken und daraus lernen
 - **Iterativ:** Den aktuellen und nächsten Schritt mit Feedback planen

Scrum: Kernelemente

- **3 Rollen:**
 - Product Owner
 - Scrum Master
 - Entwicklungsteam
- **3 Artefakte:** = zu erstellende organisator. Bausteine
 - Product Backlog
 - Sprint Backlog
 - Product Increment
- **5 Aktivitäten:** Sprint Planning, Daily Scrum, Sprint Review, Sprint-Retrospektive, Product Backlog Refinement

Aufgaben der Scrum-Rollen

- Product Owner: (PO)
 - Verantwortlich für Produkteigenschaften, wirtschaftlichen Erfolg und Ziele
 - Definiert Prioritäten für *Backlogs* in den *Sprints*
 - Erstellt *Product Backlogs* mit *Entwicklungsteam* und Stakeholdern
- Scrum Master: (SM)
 - Verantwortlich für Gelingen des Scrum, hilft bei organisatorischen Hindernissen (nicht Programmierung) oder Störungen
 - Achtet auf Einhaltung von Regeln, gibt aber keine Arbeitsanweisungen

Aufgaben der Scrum-Rollen

< 10 Mitglieder (besser 4-7), ist interdisziplinär zusammengestellt (Entwickler, Architekten, Tester, etc.)

- Entwicklungsteam: (ET)
 - Liefert Produktergebnisse nach Vorgabe des *Product Owners*
 - Wie die Backlogs umgesetzt werden, entscheidet das Team selbst
 - Im *Sprint Planning* werden Backlogs auf *Tasks* heruntergebrochen (max. 1 Tag)
 - Im *Product Backlog Refinement* werden Zeitschätzungen für die Backlogs vorgenommen
 - Das erstellte Produkt ist immer das Ergebnis des Teams, nicht von Einzelpersonen

Scrum-Aktivitäten

- Sprint Planning:
 - *PO* stellt priorisierte *Product Backlogs* vor
 - *ET* beantwortet "Was kann im nächsten Sprint bearbeitet werden?" sowie "Wie wird die Arbeit im Sprint erledigt?" (*PO* für Rückfragen dabei)
 - Kann in zwei Sitzungen aufgeteilt werden
 - Verfeinert die *Product Backlogs* in *Sprint Backlogs*
- Sprint Review:
 - *ET* stellt Ergebnisse vor
 - *PO* gleicht Ergebnisse gegen *Product Backlogs* des *Sprints* ab

Scrum-Aktivitäten

- Sprint-Retrospektive: (nach *Sprint Review*)
 - Überprüfung der eigenen Arbeitsweise ("Kann es so weitergehen?")
- Daily Scrum:
 - Zu Beginn jeden Arbeitstages als Gruppensitzung mit maximal 15 Minuten Dauer
 - SM stellt jedem Entwickler einzeln diese Fragen:
 - 1) Was haben Sie zuletzt bearbeitet?
 - 2) Gab es dabei Schwierigkeiten?
 - 3) Was planen Sie als nächstes zu tun?
 - Gegebenenfalls erfolgt eine Anpassung oder Umverteilung der Aufgaben

Scrum-Aktivitäten

- Product Backlog Refinement:
 - *PO* und *ET* zusammen:
 - Die Aufgaben werden durchgegangen:
 - Geordnet
 - Unwichtige gelöscht
 - Neue hinzufügen
 - Alte detaillieren
 - Zusammenfassen
 - Zeitschätzungen
 - Releases planen

Scrum-Artefakte

- Product Backlog:
 - Liste von Aufgaben/Anforderungen
 - Priorisiert vom *PO* (Bearbeitungsreihenfolge)
- Sprint Backlog:
 - Für *Sprint* aufbereitete *Product Backlog* Aufgabe (*item*)
 - Enthält Zeitschätzung
 - Enthält ggf. Unteraufgaben (*tasks*), so dass Zeitschätzung je *task* maximal 1 Arbeitstag
- Product-Increment:
 - Summe der fertiggestellten *Backlogs* des *Sprints*

Überblick Scrum

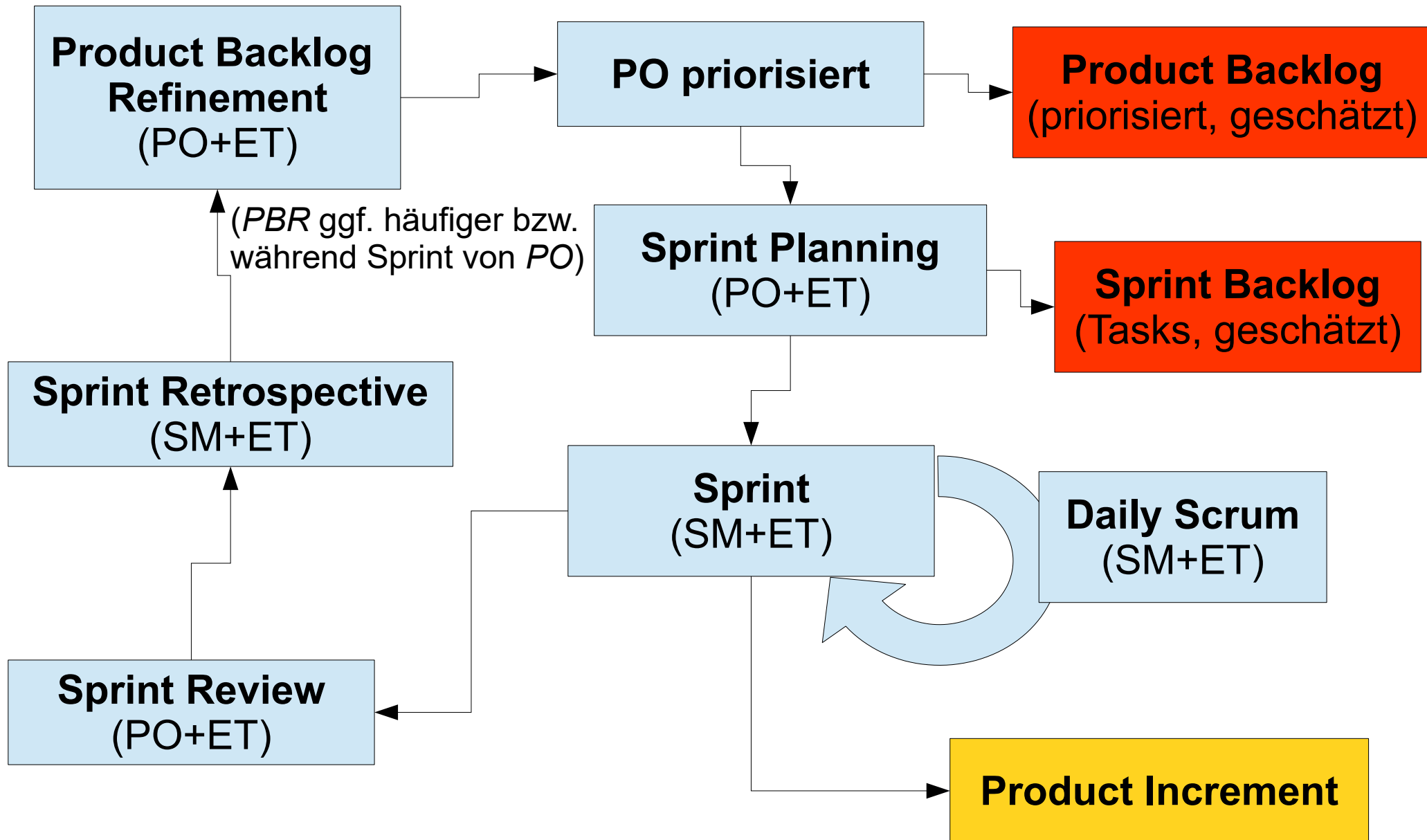
Aktivität	PO	SM	ET	Zyklus Dauer	Ergebnisse
Sprint Planning	Ja		Ja		Sprint Backlog: Sprint geplant: Was wird erledigt? Wie wird es erledigt?
Daily Scrum		Ja	Ja	1x pro Tag, max. 15 Minuten	Team auf Neuestem Stand, Hürden aus dem Weg geräumt, Plan für Weiterarbeit
Sprint Review	Ja		Ja		Stand bekannt: Backlogs geplant vs. erledigt
Sprint Retros-pektive		Ja	Ja		Arbeitsweise (für Sprint) bedacht/überarbeitet
Product Backlog Refinement	Ja		Ja		Verfeinerte Backlogs (Zeitschätzung, detaillieren, hinzufügen, löschen, ...)

Scrum: Grober Ablauf

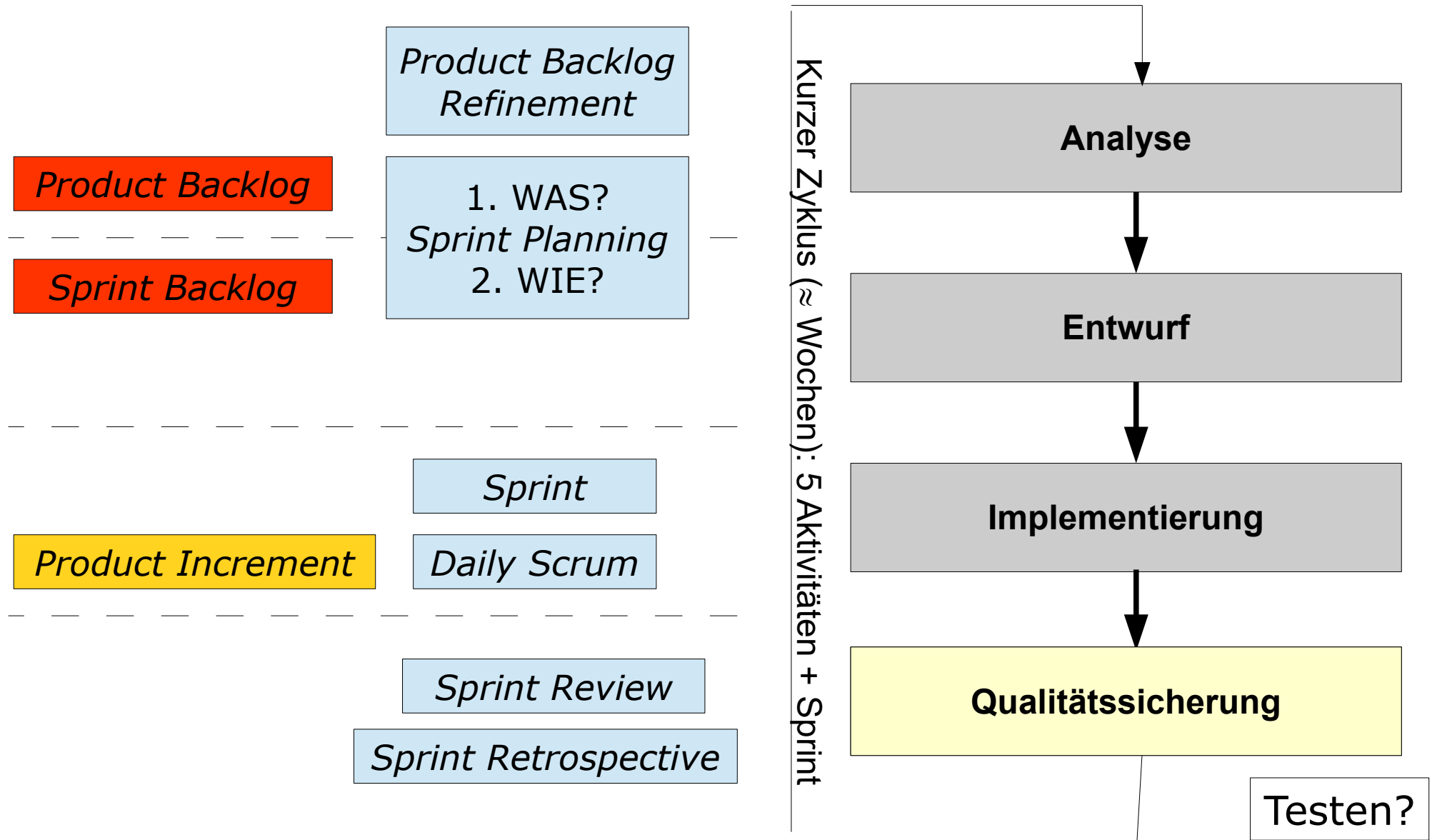
- Product Owner:
Priorisierung



Scrum Ablaufdiagramm



Scrum und Prozessphasen



Vorteile Scrum

- Hohe Flexibilität: Adaptives Planen in kurzen Zyklen
- Geringer Aufwand für Dokumentation
- Kurze und schnelle Kommunikationswege
- Hoher Grad an Selbstorganisation kann zu hoher Effizienz führen
- Hohe Transparenz durch häufige Treffen (*Daily Scrum*) und Aufgabenlisten (*Product/Sprint Backlog*)
- Zeitnahe Umsetzung von neuen Ideen und Inkrementen
- Kurzfristige Identifikation von Problemen
- Kontinuierlicher Verbesserungsprozess

Nachteile Scrum

- Hoher Kommunikationsaufwand
- Leerlauf, falls zu wenig Aufgaben je *Sprint* zugeteilt
- Systeme können schlecht strukturiert sein
(bei >100k Zeilen können schwerwiegende Probleme auftreten)
- Gesamtüberblick über Projektstruktur und Projektablauf fehlt
- Koordination von mehreren Teams bei Großprojekten ist schwierig und erfordert viel Aufwand
- Fehlende Zuständigkeiten (Hierarchien) können zu Unsicherheiten und Problemen führen
- Passung zu bisherigen Unternehmensstrukturen kann schwierig sein

Zusammenfassung

- Agile Vorgehensmodelle kennzeichnet ein schnelles Erzielen von **vorzeigbaren Ergebnissen**
 - Lauffähige Software, Demonstratoren, Prototypen
 - Vorteil: Ermöglicht gute und **schnelle Anpassung** an Kundenwünsche und wandelnde Anforderungen
 - Nachteil: Prozess sowie erstellte Systeme sind i.d.R. **unstrukturierter**, fehleranfälliger, inkompatibler mit anderen Systemen
 - Beispiele sind Evolutionäre Entwicklung, XP, Scrum, Kanban, u.A.

Fragen?